

Eks2021Opg3

April 28, 2026

1 Opgave 3

1.1 Spm. 1)

Jf. Uge 12 “Paknings-problmer” er et 01-knapsack problem defineret ved at maksimere en vægtet sum hvor beslutningsvariablene er binære 0-1 variable.

Da problemet kan formuleres

$$\max. \sum_{j=1}^n p_j \delta_j$$

s.t (1)

$$\sum_{j=1}^n w_{1j} \delta_j \leq Q_1$$

og (2)

$$\sum_{j=1}^n w_{2j} \delta_j \leq Q_2$$

$$\delta_j \in \{0, 1\} \text{ for } j = 1, \dots, n$$

hvor $w_{1j} = \{23, 19, 18, 12, 7, 5\}$ og $w_{2j} = \{8, 11, 16, 17, 14, 12\}$ må de være knapsack begrænsninger.

1.2 Spm. 2)

Lad $S \subseteq \{1, \dots, n\}$, og lad $a(S)$ være summen af koefficienter med indeks fra de respektive elementer i S , et cover er et sæt S , således at

$$a(S) > a_0$$

Hvor a_0 er upperbound fra den tilhørende knapsack ulighed.

Hvis vi tager udgangspunkt i ovenstående, samt (1), så må $S_1 = \{1, 2, 3\}$ være et cover da

$$23 + 19 + 18 > 30.$$

En cover ulighed er da en ulighed således at

$$\sum_{j \in S} \delta_j \leq |S| - 1.$$

Så det følger at en cover ulighed for (1) kan gives ved

$$\delta_1 + \delta_2 + \delta_3 \leq |S_1| - 1 = 3 - 1 = 2$$

Et minimalt cover S er defineret ved

$$a(S \setminus \{k\}) \leq a_0 \quad \forall k \in S.$$

Det følger at S_1 ikke er minimal da $a(\{1, 2\}) > 30$

Lad, $S_2 = \{1, 2\}$, som ses at være et minimalt cover da

$$a(\{1\}) = 23 \leq 30$$

og

$$a(\{2\}) = 19 \leq 30$$

Da en minimal cover ulighed er en cover ulighed for et minimalt set, er

$$\delta_1 + \delta_2 \leq |S_2| - 1 = 2 - 1 = 1$$

en minimal cover ulighed.

1.3 Spm. 3)

En udvidet cover ulighed kan generelt konstrueres ved at tilføje variable på venstresiden af en minimal cover ulighed, hvor hver tilføjede variabel har koefficient 1 og vi ikke ændrer på højresiden

En udvidet cover ulighed ECI er defineret på følgende vis. Lad S være et cover og $w^* = \max_{j \in S} w_j$, da er den Extensionen af S givet ved

$$E(S) = S \cup \{j \in \mathbb{N} \setminus S : w_j \geq w^*\}.$$

ECI er da givet ved

$$\sum_{j \in E(S)} \delta_j \leq |S| - 1.$$

Vi bemærker at ECI skal dannes fra et cover med $|S| = 3$ samt at alle koefficienter skal være med. Alle covers med kardinalitet 3 der kan dannes fra (1) ses det at der er mindst et $\{j \in \mathbb{N} \setminus S : w_j \leq w^*\}$, og derved kan (1) ikke danne udgangspunkt for den givne ECI.

Modsat, hvis vi kigger på (2), så kan ved danne et cover $S_3 = \{1, 2, 6\}$ med $|S_3| = 3$ og $w^* = 12$, hvor

$$E(S_3) = S_3 \cup \{j \in \mathbb{N} \setminus S_3 : w_j \geq w^*\} = \{1, 2, 3, 4, 5, 6\}$$

som ønsket. Derfor udgør (2) udgangspunkt for den givne ECI

1.4 Spm. 4)

FRA AI:

1. De forbedrer LP-relakseringen (afskærer fraktionelle løsninger)

Når en computer (f.eks. via en solver som Gurobi eller CPLEX) forsøger at løse et problem med 0/1-variabler, starter den næsten altid med at løse en *LP-relaksering*. Det betyder, at den ignorerer kravet om, at variablerne skal være præcis 0 eller 1, og i stedet tillader kommatall (f.eks. $x_1 = 0.5$, $x_2 = 0.8$). Udvidede cover-uligheder fungerer som effektive **snitplaner (cutting planes)**. De skærer disse “falske” kommatallsløsninger væk fra løsningsrummet uden at fjerne nogen af de gyldige heltallsløsninger.

2. De er stærkere end basis-cover-uligheder

En almindelig cover-ulighed begrænser kun de specifikke variabler, der indgår i selve coveret. Ved at udvide coveret (tilføje variabler, der er lige så store eller større), får du flere variabler ind på venstresiden af uligheden, mens højresiden (grænsen) forbliver præcis den samme ($|C| - 1$). Matematisk set betyder det, at den udvidede ulighed “dominerer” basis-uligheden. Den giver mere information til solveren på én gang.

3. De gør Branch-and-Bound meget hurtigere

Fordi den udvidede cover-ulighed skærer mere af det ubrugelige (fraktionelle) søgerum væk tidligt i processen, kommer LP-relakseringen meget tættere på den faktiske, optimale heltallsløsning. Dette betyder, at *Branch-and-Bound*-algoritmen skal undersøge langt færre grene (muligheder) i sit søgetræ. Resultatet er en massiv besparelse i både tid og computerkraft, især for store og komplekse problemer.

SLUT AI:

Problemet løses nu med LP relaksationer. Det ses at løsningen uden udvidet cover ulighed ikke finder en brugbar 0-1 løsning, men at løsningen med den udvidede cover ulighed finder en brugbar 0-1 løsning

```
[4]: import numpy as np
import pulp as PLP

# Maximisation problem
model = PLP.LpProblem("Maximisation", PLP.LpMaximize)

# Decision variables
delta_range = range(6)
delta = PLP.LpVariable.dicts("delta", delta_range, lowBound=0)

# Obj weights
p = [17,15,14,12,11,10]

# Coefficients for constraints
w1 = [23,19,18,12,7,5]
w2 = [8,11,16,17,14,12]

model += PLP.lpSum([p[j]*delta[j] for j in delta_range]), "Objective"
model += PLP.lpSum([w1[j]*delta[j] for j in delta_range]) <= 30, "Constraint1"
model += PLP.lpSum([w2[j]*delta[j] for j in delta_range]) <= 30, "Constraint2"
```

```

from CustomFunctions import print_solution
print("Løsning uden udvidet cover ulighed")
print_solution(model)

print("\nLøsning med udivdet cover ulighed")
model += PLP.lpSum([delta[j] for j in delta_range]) <= 2, "CoverInequality"

print_solution(model)

```

Løsning uden cover ulighed

```

Status: Optimal
delta_0 = 0.88983051
delta_1 = 0.0
delta_2 = 0.0
delta_3 = 0.0
delta_4 = 0.0
delta_5 = 1.9067797
Obj. = 34.19491567

```

Løsning med cover ulighed

```

Status: Optimal
delta_0 = 1.0
delta_1 = 0.0
delta_2 = 0.0
delta_3 = 0.0
delta_4 = 1.0
delta_5 = 0.0
Obj. = 28.0

```